MODULE 03 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

Provider Versioning and the Lock File

Constraints say what is acceptable. The lock file says what was chosen. Two different jobs.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 3 of 27

admin@sthithapragna.com

A constraint says what is **acceptable**. The lock file says what was **chosen**.

Providers use semantic versioning

- **MAJOR.MINOR.PATCH.** A major bump signals breaking changes, minor adds features, patch fixes bugs.
- Provider versions move fast. A widely used cloud provider may release several times a month.
- This is why constraints matter. Without one, Terraform accepts **any** version, including tomorrow's breaking release.
- The **version** argument is optional. HashiCorp strongly recommends writing one for every provider.
- Terraform selects the **newest version that satisfies every constraint** in the whole configuration, including child modules.

> ON THE EXAM
>
> Constraints from the root module and every child module are combined. All of them must be satisfied by one selected version.

The constraint operators

OPERATOR	MEANING
`= 1.4.0, or just 1.4.0`	Exactly this version. Nothing else
`!= 1.4.0`	Any version except this one
`>= 1.2.0`	This version or newer. The usual floor
`<= 1.9.0`	This version or older
`> 1.2.0 and < 2.0.0`	A range. Combine with a comma; all conditions must hold
`~> 1.4.0`	Pessimistic. Allows the rightmost part to increase, nothing above it

ON THE EXAM

Comma-separated constraints are combined with AND, never OR. Every one must be satisfied.

The pessimistic operator, and where it stops

- **~> allows the rightmost component you wrote to increase.** Everything to its left is frozen.
- **~> 1.4.0** accepts 1.4.1 and 1.4.9. It rejects 1.5.0. Patch releases only.
- **~> 1.4** accepts 1.5 and 1.9. It rejects 2.0. Minor releases allowed.
- How many components you write changes the meaning entirely. This is the detail questions are built on.
- It is **not** the same as greater than or equal to. There is always an implied ceiling.

> ON THE EXAM
>
> Count the components. Three components allow patch only; two components allow minor. Same operator, different answer.

Working an example

CONSTRAINT WRITTEN	CHOSEN FROM 1.4.0, 1.4.7, 1.5.2, 2.0.1
`~> 1.4.0`	1.4.7. Patch may rise, minor may not
`~> 1.4`	1.5.2. Minor may rise, major may not
`>= 1.4.0`	2.0.1. No ceiling at all, so the newest wins
`>= 1.4.0, < 2.0.0`	1.5.2. The comma adds a ceiling by hand
`1.4.0`	1.4.0. An exact pin, not a floor
`No version argument`	2.0.1. Any version qualifies, including breaking ones

ON THE EXAM

Terraform always takes the newest version that satisfies every constraint. The question is only ever where the ceiling sits.

required_version or required_providers?

REQUIRED_VERSION

Constrains Terraform itself

- Sits directly in the terraform block
- One setting, no source, no names
- Checked before anything else runs
- Refuses to proceed on the wrong CLI

REQUIRED_PROVIDERS VERSION

Constrains a plugin

- Sits inside required_providers
- One entry per provider, beside its source
- Resolved during terraform init
- Selects which plugin version to download

THE TIEBREAKER

Both live in the terraform block and both take the same operators. They constrain completely different software.

Where to be strict, and where not to be

- **Every module should declare at least a minimum**, using >= for the oldest version it is known to work with.
- **A root module should also set a maximum**, typically with ~>, to prevent an accidental jump to a breaking release.
- **A reusable module should not set a maximum.** HashiCorp's guidance is explicit about this.
- A ceiling inside a shared module forces every consumer to upgrade the module before they can upgrade anything else.
- Instead: state a minimum, document known incompatibilities, and **let the root module decide the ceiling**.

> ON THE EXAM
>
> The right constraint depends on where the code sits. Root modules protect themselves. Shared modules must not constrain their consumers.

The dependency lock file

- **.terraform.lock.hcl**, created and updated by **terraform init**. Terraform 0.14 onwards.
- **Commit it.** That is the point: everyone and every pipeline selects identically.
- It belongs to the **configuration as a whole**, not one per child module.
- HCP Terraform, Terraform Enterprise and the CLI all obey it.
- Do **not** hand-edit it. Manual edits may be lost, and hashes must match.

> ON THE EXAM
>
> Contrast with module two: the .terraform directory is ignored, the .terraform.lock.hcl file is committed. Similar names, opposite treatment.

Inside a lock file entry

FIELD	WHAT IT RECORDS
`Block label`	The provider's fully-qualified source address
`version`	The exact version that was selected
`constraints`	The constraint from your configuration that produced that selection
`hashes`	Checksums of the provider packages, verified on every install
`h1:` prefix	The current hashing scheme, computed over the package contents
`zh:` prefix	The legacy zip hash. One per platform the provider was built for

ON THE EXAM

Both version and constraints are recorded, so you can see the decision and the rule that produced it side by side.

What the lock file does and does not track

LOCKED

Providers

- Exact version recorded
- Checksums recorded and verified
- Same selection on every machine
- Changes only when you ask

NOT LOCKED

Modules

- No version recorded, ever
- Terraform picks newest matching the constraint
- Two machines can select differently
- Pin exactly if you need reproducibility

THE TIEBREAKER

HashiCorp is explicit: the lock file tracks only provider dependencies. Many study guides state otherwise, and they are wrong.

init, and the -upgrade flag

- **terraform init** reads the lock file first and reuses the recorded version whenever it still satisfies your constraints.
- **New providers** are resolved and added. Existing ones are left exactly where they are.
- Change a constraint so the locked version no longer fits, and init **fails** rather than silently choosing a new one.
- The error names the fix: **terraform init -upgrade**, which re-resolves every provider against the current constraints.
- That refusal is deliberate. Upgrading a provider is a decision, so Terraform makes you state it.

> ON THE EXAM
>
> Plain init is conservative by design. Upgrading within your constraints requires -upgrade, every time.

Mixed platforms, and terraform providers lock

- **init records hashes only for the platform it ran on.** Other platforms then fail.
- **terraform providers lock** writes checksums **without installing anything**.
- Use **-platform=os_arch**, once per platform your team uses.
- It also matters with a **mirror**, which never contacts the origin registry.
- Pre-recorded checksums verify the **mirror** against what the publisher signed.

> ON THE EXAM
>
> A checksum failure on a colleague's machine is usually a missing platform hash, not a corrupted provider.

When a checksum does not match

- Every install verifies the downloaded package against the hashes already in the lock file.
- **Usually benign:** a platform whose hash was never recorded. Fix with terraform providers lock and the right -platform.
- **Occasionally serious:** the package genuinely differs from the one the publisher signed.
- Never resolve it by deleting the lock file. That discards the evidence and re-records whatever you were just served.
- **terraform providers lock** prints signing information, but cannot judge whether a publisher is trustworthy.

> ON THE EXAM
>
> Deleting the lock file makes the error disappear without answering the question the error was asking.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Locked provider does not match the constraint	terraform init -upgrade
Everyone must select the same provider version	Commit the lock file
A module changed without anyone editing configuration	Modules are not locked. Pin an exact version
Checksum error only on some machines	terraform providers lock with -platform
A shared module used by many teams	Set a minimum only. No ceiling
Must never move to the next major version	The pessimistic operator, or an explicit upper bound

ON THE EXAM

Ask first whether the situation is about the acceptable range or the recorded selection. That splits nearly all of these.

> A team raises a provider constraint in required_providers from a version that has a known bug to a newer one, commits the change, and pushes.
>
> The pipeline fails during initialisation. The error states that the locked provider version does not match the configured version constraint. Nothing else in the configuration changed, and the same command worked yesterday.

What should the team do?

A. Delete .terraform.lock.hcl from the repository and re-run the pipeline

B. Run terraform init -upgrade, and commit the updated lock file

C. Add the new version to required_version in the terraform block

D. Run terraform apply -refresh=false to bypass provider resolution

◆ PAUSE AND DECIDE

What should the team do?

A Delete the lock file

Produces a working pipeline and throws away every recorded checksum. It treats a message asking for a decision as an obstacle.

B init -upgrade, then commit

Correct, and both halves matter. The flag re-resolves against the new constraint; committing the result is what makes the selection reproducible.

C Set required_version

Constrains the Terraform binary, not the provider. The pairing from earlier in this module, appearing as a distractor.

D Apply with -refresh=false

Refresh controls whether Terraform re-reads remote state before planning. It has nothing to do with provider selection, and init must succeed first anyway.

THE SIGNAL

The error message names its own fix. Terraform refuses on purpose, because changing a provider version should be deliberate.

A platform team commits .terraform.lock.hcl for every configuration and states in its standards document that all deployments are now reproducible.

Three weeks later, a plan in an untouched configuration proposes replacing a load balancer. No engineer changed any file. The configuration uses a registry module with the constraint version = "~> 4.0", and the module authors released 4.3.0 that morning.

Why did the plan change?

A The lock file recorded the module version but the constraint overrode it

B The lock file does not track modules, so Terraform selected the newest version matching ~> 4.0

C A provider upgrade was applied automatically during terraform init

D Committed lock files expire after a set period and must be regenerated

◆ PAUSE AND DECIDE

Why did the plan change?

A Constraint overrode the lock

Assumes a module entry exists in the lock file. There has never been one, so there was nothing for a constraint to override.

B Modules are not locked

Correct. The lock file covers providers only. Terraform re-resolves module versions on every init and takes the newest that fits.

C Automatic provider upgrade

Backwards. Plain init is conservative and reuses the locked provider version. That is exactly what the lock file is for.

D Lock files expire

Invents a mechanism. A lock file has no lifetime and no expiry; it changes only when a command rewrites it.

THE SIGNAL

The team's belief was half right. Providers were reproducible. Modules never were, and the standards document hid the gap.

> An engineering team is split between developers on Apple silicon laptops and a build pipeline running on Linux. Whoever runs terraform init first commits a lock file that then fails checksum verification for the other group.
>
> The team currently works around it by deleting the lock file whenever the error appears.

What should the team do instead?

A. Add .terraform.lock.hcl to .gitignore so each environment keeps its own

B. Run terraform providers lock with a -platform argument for each platform, and commit the result

C. Standardise every developer onto Linux virtual machines

D. Pin every provider to an exact version so checksums stop changing

◆ PAUSE AND DECIDE

What should the team do instead?

A Gitignore the lock file

Removes the error and the guarantee together. Each environment then selects independently, which is precisely what the file prevents.

B providers lock per platform

Correct. The command records official checksums for every platform named, without installing anything, so one committed file satisfies both groups.

C Standardise on Linux

Solves it by removing the variation, at real cost to developers, when the tool already has a purpose-built command for exactly this.

D Pin exact versions

Confuses version with platform. The version was never in dispute; the missing hash is for a different operating system and architecture.

THE SIGNAL

Option D is the trap. The mismatch is about which platforms have recorded hashes, not about which version was chosen.

> An engineer maintains a networking module published to an internal registry and used by nineteen teams. She has tested it against version 5.40 of the cloud provider and wants to prevent teams hitting breaking changes in the next major release.
>
> She proposes adding version = "~> 5.40" to the module's own required_providers block.

What should a reviewer advise?

A. Approve it. A pessimistic constraint is the correct way to prevent major-version breakage

B. Advise a minimum only, and let each root module set its own ceiling

C. Advise pinning an exact version, so all nineteen teams are guaranteed identical behaviour

D. Advise removing the version argument, since the lock file already controls selection

◆ PAUSE AND DECIDE

What should a reviewer advise?

A Approve the ceiling

Right technique, wrong place. HashiCorp advises against maximum constraints in modules intended for reuse, because the ceiling propagates to every consumer.

B Minimum only

Correct. Declare the oldest version known to work, document incompatibilities, and leave the ceiling to the root module that owns the deployment.

C Pin exactly

The strictest version of the same mistake. Nineteen teams could then never move the provider at all until the module was re-released.

D No version at all

The lock file records a selection; it does not decide which versions are acceptable. Without a constraint, any version qualifies.

THE SIGNAL

Constraints combine across every module. A ceiling in a shared module becomes a ceiling in every configuration that uses it.

Module 3 recap

- **Constraints define a range. The lock file records a selection.** Nearly every question separates on that.
- **~> freezes everything left of the component you wrote.** Count the components before answering.
- **required_version is Terraform. required_providers version is a plugin.**
- **The lock file tracks providers only.** Modules re-resolve on every init. Pin exactly if you need them still.
- **Plain init is conservative.** Moving within your constraints needs terraform init -upgrade.
- **Shared modules set a floor, never a ceiling.** Root modules set both.

> ON THE EXAM
>
> Commit .terraform.lock.hcl. Ignore the .terraform directory. Never hand-edit either.